

PROJETO SIMULADOR MIPS

Guia consolidado • requisitos • etapas • arquivos JSON • instruções

Objetivo

Desenvolver, em uma linguagem à escolha do grupo, um simulador do microprocessador MIPS estudado em sala, implementado de forma incremental em três etapas.

Este documento reúne os requisitos das três entregas, as informações adicionais (+info) e as especificações dos arquivos de entrada e saída.

Fonte principal do projeto: Genially

[Abrir projeto original no Genially](#)

Versão atualizada em setembro de 2026

1. Visão geral e navegação

O projeto está organizado em três entregas incrementais. As funcionalidades implementadas em uma etapa devem permanecer disponíveis nas etapas seguintes.

2	Requisitos gerais e organização dos arquivos
3	Entrega 1 — Identificação de instruções
4	Entrega 2 — Instruções lógicas e aritméticas
5	Entrega 3 — Load, Store e Desvio
6	Banco de Registradores
7	Memória
8	Arquivo de entrada — .json
9	Arquivo de saída — .json
10	Exemplo de saída e checklist

Fluxo recomendado

Entrada JSON → configuração inicial → leitura do código hexadecimal → decodificação → execução → atualização de registradores/memória → saída JSON.

2. Requisitos gerais e arquivos

- Implementar o simulador em uma linguagem de programação escolhida pelo grupo.
- Manter a implementação incremental: as funcionalidades anteriores continuam funcionando nas etapas seguintes.
- Trabalhar com banco de 32 registradores MIPS e com os registradores especiais PC, HI e LO.
- Implementar a memória conforme as regras da etapa correspondente.
- Respeitar rigorosamente a estrutura dos arquivos JSON de entrada e saída.
- Publicar no GitHub todos os arquivos do projeto: código-fonte, executável, arquivos de entrada e arquivos de saída.
- Informar o link do GitHub na atividade do AVA.

Critérios de avaliação

- Decodificação das instruções
- Banco de registradores
- Memória
- Conformidade
- Execução geral
- Arquivo de saída
- Comentários

Grupo

O projeto será realizado em grupos de 3 a 4 pessoas.

3. Entrega 1 — Identificador de Instruções

Objetivo: implementar a identificação/decodificação das instruções MIPS especificadas para a primeira etapa.

Entrada

- Arquivo .json contendo o código assembly para MIPS.
- As instruções devem ser processadas na mesma ordem em que aparecem no arquivo de entrada.
- Os campos regs, mem e stdout ficam vazios nesta etapa.

Saída

- Arquivo .json contendo o código em linguagem de máquina em hexadecimal.
- Cada instrução deve ser representada por um valor hexadecimal de 32 bits.

Entrada – exemplo

```
{
  "config": {},
  "data": {},
  "text": ["0x02114020"]
}
```

Decodificação

0x02114020 → add \$8, \$16, \$17

Material adicional da Entrega 1

[Abrir material adicional da Entrega 1.](#)

4. Entrega 2 — Instruções Lógicas e Aritméticas

Objetivo: executar instruções lógicas e aritméticas dos tipos R e I.

- Implementar o banco de 32 registradores e os registradores especiais PC, HI e LO.
- Após cada instrução, apresentar somente os registradores com valor diferente de zero.
- A execução de uma instrução pode alterar o resultado das instruções seguintes.
- A saída acrescenta o estado dos registradores após cada execução.

Instruções

add	sub
slt	and
or	xor
nor	mfhi
mflo	addu
subu	mult
multu	div
divu	sll
srl	sra
sllv	srlv
srav	addi
slti	andi
ori	xori
addiu	

Imagem do +info

Implementar instruções:

add
sub
slt
and
or
xor
nor
mfhi
mflo
addu
subu
mult
multu
div
divu
sll
srl
sra
sllv
srlv
sra
sllv
srlv
sra
addi
sli
andi
ori
xori
addiu

5. Entrega 3 — Load, Store e Desvio

Objetivo: implementar instruções de load, store e desvio, além da memória, mantendo as instruções das etapas anteriores.

- Para cada instrução executada, apresentar o estado da memória e do banco de registradores.
- A memória e os registradores começam zerados ou recebem os valores definidos em .config.
- A execução de uma instrução pode afetar instruções posteriores.
- As funcionalidades das etapas anteriores permanecem disponíveis.

Instruções

jr	lui
lw	sw
bltz	beq
bne	lb
lbu	sb
j	jal

Imagem do +info

Implementar instruções:

jr
lui
lw
sw
bltz
beq
bne
lb
lbu
sb
j
jal

6. Banco de Registradores

Deve ser criada uma estrutura de dados compatível com o formato do banco de registradores.

- Os registradores possuem 32 bits de tamanho, nem mais nem menos.
- A exceção gerada como resultado de operações de multiplicação e divisão são dados de 64 bits: os 32 bits mais significativos ficam armazenados em HI e os 32 bits menos significativos em LO.

Inicialização

- Os registradores devem inicializar com os mesmos valores que o MARS.
- O MARS inicia todos os valores zerados, com exceção de \$gp, \$sp e \$pc.
- \$gp = 0x10008000
- \$sp = 0x7ffefffc
- \$pc = 0x00400000
- Depois da inicialização, devem ser atribuídos os valores definidos no campo config do arquivo de entrada.

7. Memória

Deve ser criada uma estrutura de dados compatível com a representação de uma memória com dados de 8 bits.

- O MARS exibe a memória organizada em segmentos, e cada segmento possui seu próprio endereço base. Consulte os valores corretos no MARS.
- Os segmentos que devem estar no projeto são: data, sp e text.
- Cada segmento deve suportar, no mínimo, 1024 entradas de 8 bits, a partir dos endereços base.
- O simulador deve suportar endereçamento de bytes e words.

Implementação

A representação interna da memória deve permitir operações coerentes com os acessos byte e word exigidos pelas instruções da Entrega 3.

8. Arquivo de entrada (.json)

O arquivo de entrada é um JSON composto pelos campos config, data e text. Campos não utilizados devem ser representados por { }.

config

Valores carregados antes da execução das instruções.

- regs: valores pré-carregados no banco de registradores.
- Registradores na ordem correta, exatamente como no MARS; essa ordem não deve ser alterada.
- Adicionar apenas registradores com valores diferentes de zero.
- Usar nomes de registradores prefixados com \$.
- mem: endereços e valores de words pré-carregados na memória.
- Endereços em ordem crescente e valores em decimal.

data

Representa as variáveis e valores definidos no segmento .data. Os valores devem ser carregados na memória antes da execução.

text

Representa as instruções do segmento .text como um array de strings em hexadecimal.

```
{
  "config": {
    "regs": {
      "$1": 8,
      "$2": 12,
      "$29": 2147489648,
      "$31": 4195360,
      "pc": 4194400,
      "lo": 100
    },
    "mem": {
      "2147479599": 0,
      "2147479574": 4194376,
      "2147479508": 1,
      "2147479882": 4194376
    }
  },
  "data": {
    "268500992": "192029",
    "268508174": "4990970",
    "268502144": "4990977"
  },
  "text": [
    "0x24020004", "0x3c011001", "0x3424006b",
    "0x3424006b", "0x3424006b", "0x3424006b"
  ]
}
```

9. Arquivo de saída (.json)

Para cada instrução presente no array `text` do arquivo de entrada, a saída deve conter os campos `hex`, `text`, `regs`, `mem` e `stdout`. Campos não utilizados devem ser representados por `{ }`.

hex

Instrução em hexadecimal correspondente à instrução do arquivo de entrada.

text

Código assembly MIPS correspondente à decodificação do hexadecimal.

regs

- Instantâneo do banco de registradores imediatamente após a execução.
- Ordem: `$0` a `$31`, seguidos por `pc`, `hi` e `lo`.
- Somente registradores com dados diferentes de zero devem ser apresentados.
- Registradores especiais: `$pc`, `$hi` e `$lo`.

mem

- Instantâneo da memória imediatamente após a execução.
- Endereços e dados em decimal, com endereços em ordem crescente.
- Somente endereços com dado diferente de zero devem ser apresentados.

stdout

Indica ocorrência de overflow: usar `"overflow"` quando ocorrer; caso contrário, deixar o campo vazio.

10. Exemplo de saída e checklist

```
{
  "hex": "0x0810004a",
  "text": "j 1048650",
  "regs": {
    "$9": 3,
    "$10": 4,
    "$11": -3,
    "$12": -4,
    "$13": 2147483647,
    "$14": -2147483648,
    "$15": 4194608,
    "$16": 4456448,
    "$17": 268500992,
    "$18": 268500996,
    "$19": 268501500,
    "$20": -1000,
    "$28": 268468224,
    "$29": 2147479548,
    "pc": 4194600,
    "hi": 1,
    "lo": 1
  },
  "mem": {
    "268500992": 4194608,
    "268500996": -2147483648,
    "268501500": 4194608
  },
  "stdout": "overflow"
}
```

Checklist de conformidade

- JSON válido e com os campos corretos.
- Não incluir registradores zerados quando a regra exigir apenas valores diferentes de zero.
- Manter a ordem dos registradores exatamente como especificado.
- Ordenar endereços de memória em ordem crescente.
- Atualizar o estado após cada instrução.
- Preservar as funcionalidades das etapas anteriores.
- Usar overflow em stdout quando houver overflow.

Referências

- [Projeto original no Genially](#)
- [Material adicional da Entrega 1](#)